# 📚 DAY 5: MASTER TERMINOLOGY & CONCEPT GLOSSARY

## Generative AI, Large Language Models (LLMs), Habana Gaudi & OpenVINO Acceleration

### Comprehensive Hardware, Algorithmic & Systems Lexicon for Trainers and Students

**Course Code:** GLA-HPC-BCSG701 | **Track:** Intel & NEC Supercomputing Alliance

## 📌 TABLE OF CONTENTS

1. Core Generative AI & Transformer Architectures (Terms 1–8)
2. LLM Inference Dynamics & The Memory Wall (Terms 9–14)
3. Intel Silicon for LLMs: Xeon AMX & Habana Gaudi (Terms 15–22)
4. Intel OpenVINO Toolkit & Model Optimization (Terms 23–29)
5. Distributed Fine-Tuning & Parameter-Efficient Tuning (PEFT) (Terms 30–36)

## 1. Core Generative AI & Transformer Architectures

### 01. Large Language Model (LLM)

- **Conceptual Definition:** Deep neural network architectures composed of stacked multi-head self-attention layers and feed-forward networks (FFNs) with billions of parameters, trained on massive tokenized text corpora to model language probability distributions.
- **Underlying HPC Mechanism:** Compute demands scale quadratically with context length ($O(L^2)$) in standard self-attention, and model parameter sizes exceed the single-socket memory limits of conventional servers, requiring specialized tensor parallelism (TP), pipeline parallelism (PP), and distributed memory architectures.
- **Trainer Cue:** "*Remind students that an LLM is not magic; it is millions of matrix multiplications computing conditional probability distributions over a fixed vocabulary.*"

### 02. Auto-Regressive Decoding

- **Conceptual Definition:** The text generation process where tokens are emitted sequentially, one by one. Each newly generated token is appended to the input sequence and fed back into the model to predict the subsequent token.
- **Underlying HPC Mechanism:** Generates two distinct operational phases: the **Prefill Phase** (parallel processing of the prompt, highly compute-bound) and the **Decode Phase** (sequential single-token emission, severely memory-bandwidth bound).
- **Trainer Cue:** "*Highlight the contrast: Prefill is a massive parallel matrix multiplication, while Decode is a repeated memory sweep for a single token.*"

## 03. Multi-Head Self-Attention (MHA)

- **Conceptual Definition:** The core mechanism of the Transformer architecture where input embeddings are projected into Query ($Q$), Key ($K$), and Value ($V$) matrices across $H$ independent attention heads to capture disparate semantic relationships.
- **Underlying HPC Mechanism:** Evaluates the attention equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  On Intel hardware, the batched matrix multiplications ($QK^T$ and $\text{Attn} \cdot V$) are accelerated via Intel AMX (Advanced Matrix Extensions) and oneDNN GEMM primitives.
- **Trainer Cue:** "*Point out that multi-head attention splits channel dimensions across heads, allowing parallel GEMMs on separate vector/matrix units.*"

## 04. Multi-Query Attention (MQA) & Grouped-Query Attention (GQA)

- **Conceptual Definition:** Architectural optimizations that compress the Key and Value heads while retaining multiple Query heads. MQA shares a single Key/Value head across all Query heads; GQA groups Query heads into clusters sharing a Key/Value head.
- **Underlying HPC Mechanism:** Drastically reduces the memory footprint and DRAM bandwidth requirements of the KV-Cache by $8\times$ to $16\times$, resolving the memory bandwidth choke during auto-regressive generation.
- **Trainer Cue:** "*Show Llama-3 and Mistral architectures as prime real-world examples of GQA delivering high TPS with reduced cache overhead.*"

## 05. Tokenization & Vocabulary Embeddings

- **Conceptual Definition:** The process of decomposing unstructured raw natural language text into discrete integer token IDs using algorithms such as Byte-Pair Encoding (BPE) or WordPiece.
- **Underlying HPC Mechanism:** Token IDs index directly into a large embedding table (e.g. $128\text{k} \times 4096$), performing a memory gather operation to yield dense vector representations.
- **Trainer Cue:** "*Clarify that tokens do not map 1:1 to words; in English, 1,000 words typically equate to ~1,333 tokens.*"

## 06. Rotary Position Embedding (RoPE)

- **Conceptual Definition:** A relative positional encoding method that encodes token position by rotating the Query and Key vectors in the complex 2D plane rather than adding absolute positional vectors.
- **Underlying HPC Mechanism:** Employs element-wise trigonometric rotation kernels. Highly vectorizable on Intel AVX-512 and Habana Gaudi TPC (Tensor Processor Core) engines.
- **Trainer Cue:** "*RoPE enables natural context window extrapolation without retraining base weights.*"

## 07. Feed-Forward Network (FFN / SwiGLU)

- **Conceptual Definition:** The non-linear transformation layer following self-attention in a Transformer block, typically expanding the hidden dimension to $\frac{8}{3}d_{\text{model}}$ before projecting back down.
- **Underlying HPC Mechanism:** Represents over 60% of the total floating-point operations in modern LLMs. Uses SwiGLU activation ($\text{Swish}(xW_1) \otimes (xW_3)$), executed via fused GEMM-elementwise kernels in OpenVINO.
- **Trainer Cue:** "*Stress that FFN layers contain the majority of model parameters, making them prime targets for weight quantization.*"

### 08. Context Length / Window ($L$)

- **Conceptual Definition:** The maximum number of continuous tokens (prompt + generated response) that the model can maintain in its active attention span.
- **Underlying HPC Mechanism:** Memory consumed by activations and the KV-Cache scales linearly with context length ($O(L)$), causing out-of-memory (OOM) crashes if memory allocation is not strictly bounded.
- **Trainer Cue:** "*Ask students: What happens to cluster RAM when 60 concurrent users submit 32k context prompts simultaneously?*"

---

## 2. LLM Inference Dynamics & The Memory Wall

### 09. The Memory Wall in LLM Inference

- **Conceptual Definition:** The fundamental architectural disconnect where CPU/GPU arithmetic processing speed (TFLOPs) has outpaced main memory bandwidth (GB/s) by orders of magnitude over the past two decades.
- **Underlying HPC Mechanism:** In auto-regressive decoding, to generate a single token, all model weights (e.g., 7 billion FP16 parameters = 14 GB) must be transferred from RAM into processor caches to perform a single vector-matrix product ($O(1)$ arithmetic intensity). Execution is 100% bound by memory bandwidth.
- **Trainer Cue:** "*Write on the board: Arithmetic Intensity of Decode = 1 FLOP / 2 Bytes = 0.5. It is impossible to be compute-bound during single-token decoding!*"

### 10. Key-Value Cache (KV-Cache)

- **Conceptual Definition:** A memory buffer that stores the computed Key and Value projection matrices for all historical tokens in the sequence, preventing redundant recomputation during auto-regressive decoding.
- **Underlying HPC Mechanism:** Eliminates $O(L^2)$ redundant recomputation at the expense of expanding dynamic memory:

$$\text{KV-Cache Memory (Bytes)} = 2 \times B \times L \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{bytes_per_elem}$$

- **Trainer Cue:** "*Explain that KV-cache trades memory consumption for computational speed.*"

### 11. Prefill Phase (Time to First Token - TTFT)

- **Conceptual Definition:** The initial stage of LLM inference where the entire input prompt is processed concurrently in a single forward pass to populate the initial KV-Cache and emit the first response token.
- **Underlying HPC Mechanism:** Highly compute-bound ($AI \gg 10\ \text{FLOP/byte}$). Saturated with large dense matrix multiplications that fully exploit Intel AMX and AVX-512 systolic execution.
- **Trainer Cue:** "*TTFT measures latency perceived by the user before output starts streaming.*"

### 12. Decode Phase (Tokens Per Second - TPS)

- **Conceptual Definition:** The ongoing auto-regressive generation phase where the model emits tokens one at a time until reaching an end-of-sequence (`<|im_end|>`) token or maximum length limit.
- **Underlying HPC Mechanism:** Strictly memory-bandwidth bound. Performance is directly proportional to DRAM / HBM memory transfer rate.
- **Trainer Cue:** "*TPS dictates how fast the model types out text to the user.*"

### 13. PagedAttention & vLLM Architecture

- **Conceptual Definition:** A memory management algorithm inspired by virtual memory paging in operating systems, storing continuous KV-cache sequences in non-contiguous physical memory blocks.
- **Underlying HPC Mechanism:** Reduces dynamic KV-cache memory waste from internal and external fragmentation from 60–80% down to under 4%, enabling $2\times$ to $4\times$ higher concurrent request batching.
- **Trainer Cue:** "*Compare PagedAttention to OS virtual memory page tables allocating physical frames.*"

### 14. Speculative Decoding

- **Conceptual Definition:** An inference acceleration technique where a small, fast draft model generates candidate token sequences, which are then validated in a single forward pass by the large target model.
- **Underlying HPC Mechanism:** Converts multiple memory-bound single-token decode passes into a single compute-bound verification pass, increasing TPS by $2\times$ to $3\times$ without altering output distribution.
- **Trainer Cue:** "*Speculative decoding leverages idle compute units during memory-bound decoding.*"

---

## 3. Intel Silicon for LLMs: Xeon AMX & Habana Gaudi

### 15. Intel Advanced Matrix Extensions (AMX)

- **Conceptual Definition:** Dedicated hardware matrix acceleration engines integrated directly into each core of 4th Gen (Sapphire Rapids) and 5th Gen (Emerald Rapids) Intel Xeon Scalable processors.
- **Underlying HPC Mechanism:** Introduces two primary hardware components: **Tiles** (8 1KB 2D register files `TMM0` – `TMM7`) and **TMUL** (Tile Matrix Multiply Unit), capable of computing $1024$ INT8 operations or $512$ BF16 operations per single clock cycle.
- **Trainer Cue:** "*AMX is not an instruction set extension like AVX; it is a true systolic 2D matrix engine on the CPU die.*"

### 16. Tile Registers ( `TMM0` – `TMM7` )

- **Conceptual Definition:** Eight physical 2D registers in each CPU core, each holding a $16 \times 64$ byte matrix (1024 bytes per tile, 8KB total state per core).
- **Underlying HPC Mechanism:** Configured via Linux kernel system calls ( `arch_prctl(ARCH_REQ_XCOMP_PERM)` ). Operates on continuous row-major sub-matrices loaded directly from L1/L2 cache.
- **Trainer Cue:** "*Highlight that software must explicitly request AMX tile permissions from the Linux kernel before loading TMM registers.*"

### 17. Bfloat16 (Brain Floating Point) Format

- **Conceptual Definition:** A 16-bit floating-point numeric format consisting of 1 sign bit, 8 exponent bits, and 7 mantissa (fraction) bits, matching the dynamic range of FP32.
- **Underlying HPC Mechanism:** Eliminates underflow/overflow scaling issues common to IEEE FP16 without requiring loss-scaling hyperparameter tuning. Fully supported in hardware by Intel AMX and Habana Gaudi.
- **Trainer Cue:** "*Bfloat16 has the same exponent bits as FP32, so dynamic range is identical while cutting memory bandwidth in half.*"

### 18. Habana Gaudi 2 Architecture

- **Conceptual Definition:** A high-performance AI training and inference accelerator designed by Habana Labs (an Intel company), built specifically for deep learning tensor contractions and large-scale cluster scaling.

- **Underlying HPC Mechanism:** Integrates 24 fully programmable **Tensor Processor Cores (TPC)**, 2 Matrix Multiplication Engines (MME), and 96 GB of ultra-fast HBM2e memory delivering 2.45 TB/s bandwidth.
- **Trainer Cue:** "*Point out that Gaudi 2 delivers competitive performance against NVIDIA A100 at significantly lower Total Cost of Ownership (TCO).*"

### 19. Habana Gaudi 3 Architecture

- **Conceptual Definition:** Intel's next-generation AI accelerator manufactured on TSMC 5nm, featuring 64 TPCs, 8 MMEs, and 128 GB of HBM3 memory providing 3.7 TB/s bandwidth.
- **Underlying HPC Mechanism:** Delivers 1,835 TFLOPs of FP8 compute power and integrates 24 on-die 200GbE RoCE network ports for massive supercomputing clusters.
- **Trainer Cue:** "*Emphasize that Gaudi 3 doubles FP8 compute and upgrades on-die networking to 200GbE per link.*"

### 20. Matrix Multiplication Engine (MME)

- **Conceptual Definition:** The fixed-function systolic array hardware inside Habana Gaudi dedicated exclusively to performing large-scale 2D and 3D dense matrix multiplications and convolutions.
- **Underlying HPC Mechanism:** Executes GEMM operations with peak throughput, offloading intensive matrix math from the programmable TPC cores.
- **Trainer Cue:** "*MME handles the heavy dense matrix contractions, while TPCs execute custom activations and non-linear ops.*"

### 21. Tensor Processor Core (TPC)

- **Conceptual Definition:** A custom Very Long Instruction Word (VLIW) SIMD processor core optimized for non-linear deep learning operations (LayerNorm, Softmax, GeLU, RoPE).
- **Underlying HPC Mechanism:** Features a 2048-bit wide vector processing unit, unrolled execution, and direct hardware memory access without OS kernel interruption.
- **Trainer Cue:** "*Engineers write custom TPC C kernels when creating novel non-linear activation functions.*"

### 22. On-Die 24x 100/200GbE RoCE Networking

- **Conceptual Definition:** Integrated network controllers built directly onto the Gaudi silicon die supporting RDMA over Converged Ethernet (RoCE v2).
- **Underlying HPC Mechanism:** Eliminates the need for discrete external PCIe network interface cards (NICs). Standard cost-effective Ethernet switches interconnect Gaudi nodes without expensive proprietary InfiniBand switches.
- **Trainer Cue:** "*This is Gaudi's crown jewel: networking is integrated directly into the processor die, drastically cutting cluster cost.*"

---

## 4. Intel OpenVINO Toolkit & Model Optimization

### 23. Intel OpenVINO Toolkit

- **Conceptual Definition:** An open-source suite of developer tools and runtime libraries developed by Intel for high-performance optimization, quantization, and deployment of AI models across CPUs, GPUs, and NPUs.
- **Underlying HPC Mechanism:** Ingests models from PyTorch, ONNX, and TensorFlow, applies graph-level optimizations (operator fusion, constant folding, dead-code elimination), and executes via hardware-specific execution plugins.
- **Trainer Cue:** "*Write Once, Deploy Anywhere across Intel CPUs, integrated GPUs, discrete Arc GPUs, and NPUs.*"

## 24. Intermediate Representation (IR)

- **Conceptual Definition:** OpenVINO's native model format representing the deep learning computation graph, split into two complementary files:
  1. `model.xml`: XML document defining graph topology, layer connections, shapes, and data types.
  2. `model.bin`: Contiguous binary file storing constant weights and biases.
- **Underlying HPC Mechanism:** Allows rapid memory-mapped ( `mmap` ) loading of model weights directly into processor address space without Python interpreter overhead.
- **Trainer Cue:** "*XML holds the blueprint; BIN holds the raw weights.*"

## 25. Neural Network Compression Framework (NNCF)

- **Conceptual Definition:** Intel's open-source library for advanced post-training quantization, pruning, and sparsity optimization of deep learning models.
- **Underlying HPC Mechanism:** Analyzes weight matrices and activation distributions to convert 32-bit/16-bit floating point representations into low-bit integer formats (INT8, INT4) while maintaining accuracy within 99% of baseline.
- **Trainer Cue:** "*NNCF provides both Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT).*"

## 26. Weight-Only Quantization (INT4 / INT8)

- **Conceptual Definition:** A compression method tailored specifically for memory-bound LLM decoding where only linear layer weights are quantized to 4-bit or 8-bit integers, while activation tensors remain in FP16/BF16.
- **Underlying HPC Mechanism:** During decode passes, 4-bit weights are read from DRAM ($4\times$ less data moved!), dequantized on-the-fly inside on-chip cache, and multiplied against activations.
- **Trainer Cue:** "*Explain why activations aren't INT4 in basic inference: weight movement dominates decoding time, so quantizing weights alone yields nearly full speedup.*"

## 27. Asymmetric vs. Symmetric Quantization

- **Conceptual Definition:** Quantization mapping schemes from real floating-point values to discrete integers.
  - **Symmetric:** Zero in real space maps strictly to zero in integer space ($\text{Zero Point} = 0$).
  - **Asymmetric:** Uses both a scale factor ($S$) and an integer zero-point ($Z$) offset ($x_q = \text{round}(x/S) + Z$).
- **Underlying HPC Mechanism:** Asymmetric quantization provides higher accuracy for asymmetric weight/activation distributions (e.g. ReLU, GeLU) at the expense of an extra subtraction during dequantization.
- **Trainer Cue:** "*Asymmetric captures skewed distributions with less quantization noise.*"

## 28. Operator Fusion

- **Conceptual Definition:** A compiler graph transformation that merges multiple adjacent operations into a single combined execution kernel (e.g., Conv + Bias + ReLU, or QKV Projection + RoPE).
- **Underlying HPC Mechanism:** Eliminates round-trip memory writes and reads between on-chip registers and external DRAM. Intermediate results remain hot inside L1/L2 cache registers throughout the kernel.
- **Trainer Cue:** "*Operator fusion saves memory bandwidth by keeping intermediate values inside registers.*"

## 29. `ov::genai::LLMPipeline`

- **Conceptual Definition:** OpenVINO's optimized C++ inference runtime engine designed specifically for generative language models, featuring built-in tokenizers, streaming, and PagedAttention KV-caching.
- **Underlying HPC Mechanism:** Bypasses Python runtime bottlenecks entirely, delivering sub-millisecond per-token dispatch on Intel CPUs and GPUs.

- **Trainer Cue:** "*Show how 4 lines of Python code using LLMPipeline replace 50 lines of boilerplate HuggingFace code.*"

---

## 5. Distributed Fine-Tuning & Parameter-Efficient Tuning (PEFT)

### 30. Parameter-Efficient Fine-Tuning (PEFT)

- **Conceptual Definition:** A methodology for adapting pre-trained foundation models to specific downstream domains by freezing the majority of base weights and updating only a tiny fraction ($< 1\%$) of parameters.
- **Underlying HPC Mechanism:** Slashes cluster VRAM requirements during backpropagation. Optimizer states and gradients are only computed and stored for the small active parameter subset, preventing cluster memory exhaustion.
- **Trainer Cue:** "*PEFT makes fine-tuning 70B models possible on accessible cluster nodes.*"

### 31. Low-Rank Adaptation (LoRA)

- **Conceptual Definition:** A PEFT technique that decomposes weight updates into two low-rank matrices: $\Delta W = B \cdot A$, where $W_0 \in \mathbb{R}^{d \times k}$, $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, with rank $r \ll \min(d, k)$.
- **Underlying HPC Mechanism:**

$$W = W_0 + \frac{\alpha}{r}(B \cdot A)$$

  During training, base weights $W_0$ remain frozen. Only matrices $A$ and $B$ receive gradients, reducing trainable parameters by over $99\%$.
- **Trainer Cue:** "*Derive on the whiteboard: If $d = 4096$ and $r = 8$, parameters drop from $4096 \times 4096 = 16.7\text{M}$ down to $2 \times 4096 \times 8 = 65.5\text{K}$!*"

### 32. Quantized Low-Rank Adaptation (QLoRA)

- **Conceptual Definition:** An extension of LoRA where the frozen base model weights are quantized into a specialized 4-bit NormalFloat (NF4) data type, while LoRA adapters remain in 16-bit precision.
- **Underlying HPC Mechanism:** Enables fine-tuning large 65B/70B parameter models on a single high-memory node or distributed multi-socket Xeon cluster without out-of-memory errors.
- **Trainer Cue:** "*QLoRA combines INT4 weight compression with LoRA adapter agility.*"

### 33. LoRA Rank ($r$) & Scaling Factor ($\alpha$)

- **Conceptual Definition:** Hyperparameters controlling the rank and magnitude of the adapter matrices. Rank $r$ defines the dimension of the low-rank bottleneck; $\alpha$ scales the adapter output.
- **Underlying HPC Mechanism:** Higher rank $r$ increases representational capacity and adapter memory footprint; typical values are $r \in [8, 16, 64]$ with $\alpha = 2 \times r$.
- **Trainer Cue:** "*Explain that for most language tasks, $r = 8$ or $16$ captures almost all domain-specific performance.*"

### 34. DeepSpeed ZeRO (Zero Redundancy Optimizer)

- **Conceptual Definition:** A distributed memory optimization system developed by Microsoft that partitions optimizer states, gradients, and model parameters across data-parallel processes.
- **Underlying HPC Mechanism:**
  - **ZeRO-Stage 1:** Partitions optimizer states ($4\times$ memory reduction).
  - **ZeRO-Stage 2:** Partitions optimizer states + gradients ($8\times$ memory reduction).

    - **ZeRO-Stage 3:** Partitions optimizer states + gradients + model parameters ($N\times$ memory reduction across $N$ nodes).
- **Trainer Cue:** "*ZeRO-3 eliminates parameter duplication entirely across cluster nodes.*"

### 35. HuggingFace Optimum Intel

- **Conceptual Definition:** An open-source acceleration library that bridges HuggingFace Transformers with Intel hardware engines (OpenVINO, oneDNN, Habana SynapseAI).
- **Underlying HPC Mechanism:** Injects optimized Intel kernels directly into standard HuggingFace training and inference pipelines ( `AutoModelForCausalLM.from_pretrained(..., backend="openvino")` ).
- **Trainer Cue:** "*Optimum Intel gives students a familiar HuggingFace interface with oneAPI speed under the hood.*"

### 36. SynapseAI Software Suite

- **Conceptual Definition:** Habana's enterprise software stack that compiles, optimizes, and executes deep learning models on Habana Gaudi hardware.
- **Underlying HPC Mechanism:** Includes the SynapseAI Graph Compiler, TPC development tools, Communication Libraries (HCCL - Habana Collective Communications Library), and PyTorch runtime plugins.
- **Trainer Cue:** "*SynapseAI is to Gaudi what CUDA/NCCL is to NVIDIA.*"

---

*GLA University HPC Masterclass | Day 5 Master Terminology Completed*